Lab 4: Weather Stations Monitoring — Technical Report

Course: Net Centric Computing (Fall 2025–2026)

Names: Abdullah Amgad 8714

Tarek Mohamed 8924

Belal Mohamed 8906

Abdelrahman Ayman 8725

1. Overview & System Architecture

The Internet of Things (IoT) generates massive data streams at high frequencies, requiring efficient distributed stream processing. This lab implements a distributed weather monitoring system consisting of three core layers:

- **Data Acquisition:** Multiple mock weather station services emit periodic weather status readings to Apache Kafka topics.
- **Data Processing:** Kafka brokers and stream processors handle ingestion, filtering, and condition detection (such as rain triggers when humidity exceeds 70%).
- **Data Persistence & Analytics:** A Central Station consumes the data streams and executes batch inserts into a relational SQL database (PostgreSQL), which serves as the analytical data store.

[10 Weather Stations] ---> (Kafka Broker / Zookeeper) ---> [Central Station / Rain Processor] ---> [(PostgreSQL Database)]

2. System Architecture & Components

2.1 Data Acquisition (Weather Stations)

- **Replicas:** Deployed as a Kubernetes deployment scaling 10 independent mock weather station pods.
- **Frequency:** Each station outputs a status message every 1 second.
- **Probability & Error Simulation:**
    - Battery status distribution: ~30% Low, ~40% Medium, ~30% High.
    - Network reliability: Simulated random message drop rate of ~10%.
- JSON Schema Format:


```
{
 "station_id": 1,
 "sequence_number": 1,
 "battery_status": "low",
 "timestamp": 1787593482,
 "weather": {
  "humidity": 67,
  "temperature": 20,
  "wind_speed": 66
 }
}
```

2.2 Data Processing & Brokerage

- **Kafka & Zookeeper:** Deployed inside the Kubernetes cluster to manage message routing, topics (weather_data), and consumer coordination.
- **Rain Processor:** An independent service processing stream items to detect adverse conditions (humidity > 70%).

2.3 Data Persistence (SQL Database)

- **PostgreSQL Database:** Stores historical records using batch writes managed by the Central Station to prevent excessive I/O overhead.
- Table Schema:

```
CREATE TABLE weather_readings (
    id BIGSERIAL PRIMARY KEY,
    station_id BIGINT,
    sequence_number BIGINT,
    battery_status VARCHAR(10),
    timestamp BIGINT,
    humidity INT,
    temperature INT,
    wind_speed INT
);
```

3. Kubernetes Deployment

To transition from local testing to a scalable production-like environment, the architecture was fully containerized via Dockerfiles and orchestrated using Minikube Kubernetes manifests.

3.1 Cluster Manifest Summary

- **Infrastructure:** Deployment pods for Zookeeper, Kafka, and PostgreSQL with persistent storage volumes.
- **Applications:** Deployments for the Central Station, Rain Processor, and 10 Weather Station replicas.

4. Historical Analysis & SQL Validation

To validate system reliability, data integrity, and compliance with lab specifications, data accumulated in PostgreSQL was queried using custom SQL aggregations.

4.1 Battery Status Distribution Analysis

The target specification required a randomized distribution of 30% Low, 40% Medium, and 30% High battery reporting across stations.

SQL Query Used:

```
SELECT
  CAST(station_id AS TEXT) AS station,
  ROUND(COUNT(CASE WHEN battery_status = 'low' THEN 1 END) * 100.0 / COUNT(*), 2) AS low_pct,
  ROUND(COUNT(CASE WHEN battery_status = 'medium' THEN 1 END) * 100.0 / COUNT(*), 2) AS
medium_pct,
  ROUND(COUNT(CASE WHEN battery_status = 'high' THEN 1 END) * 100.0 / COUNT(*), 2) AS high_pct
FROM weather_readings
GROUP BY station_id
UNION ALL
SELECT
  'OVERALL AVG' AS station,
  ROUND(COUNT(CASE WHEN battery_status = 'low' THEN 1 END) * 100.0 / COUNT(*), 2),
  ROUND(COUNT(CASE WHEN battery_status = 'medium' THEN 1 END) * 100.0 / COUNT(*), 2),
  ROUND(COUNT(CASE WHEN battery_status = 'high' THEN 1 END) * 100.0 / COUNT(*), 2)
FROM weather_readings
ORDER BY station;
```

Results Table:

```
   station   | low_pct | medium_pct | high_pct
-------------+---------+------------+----------
 25          |   29.87 |      41.95 |    28.19
 26          |   27.95 |      42.76 |    29.29
 27          |   34.88 |      40.20 |    24.92
 28          |   28.66 |      39.41 |    31.92
 29          |   31.60 |      42.01 |    26.39
 30          |   28.15 |      45.03 |    26.82
 31          |   31.35 |      37.95 |    30.69
 32          |   30.36 |      38.28 |    31.35
 33          |   22.79 |      44.22 |    32.99
 34          |   28.34 |      40.07 |    31.60
 OVERALL AVG |   29.40 |      41.17 |    29.43
(11 rows)
```

4.2 Message Drop Rate & Reliability Analysis

To verify network reliability under the simulated 10% packet-loss constraint, expected sequence limits were contrasted against actual rows ingested.

SQL Query Used:

```
SELECT
   CAST(station_id AS TEXT) AS station,
   MAX(sequence_number) AS expected_msgs,
   COUNT(*) AS received_msgs,
   (MAX(sequence_number) - COUNT(*)) AS dropped_msgs,
   ROUND((MAX(sequence_number) - COUNT(*)) * 100.0 / MAX(sequence_number), 2) AS drop_rate_pct
FROM weather_readings
GROUP BY station_id
UNION ALL
SELECT
   'OVERALL AVG' AS station,
   CAST(AVG(exp_cnt) AS INTEGER),
   CAST(AVG(rec_cnt) AS INTEGER),
   CAST(AVG(drop_cnt) AS INTEGER),
   ROUND(AVG(drop_pct), 2)
FROM (
   SELECT
      station_id,
      MAX(sequence_number) AS exp_cnt,
      COUNT(*) AS rec_cnt,
      (MAX(sequence_number) - COUNT(*)) AS drop_cnt,
      (MAX(sequence_number) - COUNT(*)) * 100.0 / MAX(sequence_number) AS drop_pct
   FROM weather_readings
   GROUP BY station_id
) sub
ORDER BY station;
```

Results Table:

```
   station    | expected_msgs | received_msgs | dropped_msgs | drop_rate_pct
--------------+---------------+---------------+--------------+---------------
 25           |           334 |           298 |           36 |         10.78
 26           |           335 |           297 |           38 |         11.34
 27           |           335 |           301 |           34 |         10.15
 28           |           335 |           307 |           28 |          8.36
 29           |           335 |           288 |           47 |         14.03
 30           |           335 |           302 |           33 |          9.85
 31           |           335 |           303 |           32 |          9.55
 32           |           335 |           303 |           32 |          9.55
 33           |           336 |           294 |           42 |         12.50
 34           |           335 |           307 |           28 |          8.36
 OVERALL AVG  |           335 |           300 |           35 |         10.45
(11 rows)
```

5. Conclusion

The distributed weather monitoring system was successfully implemented, containerized, and deployed on a Kubernetes cluster via Minikube. All streaming routines, Kafka messaging pipelines, batch database commits, and analytical verifications met the functional requirements outlined for Lab 4.